Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра **гуманитарных, математических и естественно научных дисциплин**

ПРАКТИЧЕСКАЯ РАБОТА

Дисциплина(модуль)/предмет **МДК.02.02**

Тема: **Разработка перечня артефактов и протоколов проекта**

ВАРИАНТ № **3**

Студент (ка)/учащийся: **Браунков Матвей Константинович** / _____ /
Ф. И. О. подпись

Группа / Класс И14-23-9

Руководитель: **Чудинова А.А.** _____ /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «____» _____ 202__г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 АРТЕФАКТЫ ПРОЕКТА.....4
2 ПРОТОКОЛ РАБОТЫ С GIT И CODE REVIEW.....5
3 АРХИТЕКТУРНОЕ РЕШЕНИЕ ADR-001.....6
ЗАКЛЮЧЕНИЕ.....7
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....8

ВВЕДЕНИЕ

Цель работы: составить паспорт артефактов и протоколов проекта – определить, какие документы и файлы сопровождают разработку, как команда работает с Git и как фиксируются архитектурные решения.

Вариант 3: АИС «Логистический хаб». Система отслеживает перемещение посылок, принимает сканирование QR-кодов и RFID-меток на складах и строит маршруты доставки. Паспорт состоит из трёх разделов: артефакты проекта, протокол работы с Git и код-ревью, архитектурное решение ADR-001.

Таблица 1 – Сведения о работе

Параметр	**Значение**
Вариант №	3
Название АИС	Логистический хаб
Студент	Браунков Матвей Константинович
Группа	И14-23-9

1 АРТЕФАКТЫ ПРОЕКТА

Артефакт – это любой результат работы, который хранится и используется командой. Для АИС «Логистический хаб» выбраны четыре артефакта: по одному каждого вида.

Таблица 2 – Перечень артефактов проекта

Вид	**Артефакт**	**Назначение в АИС**	**Формат и место хранения**
Архитектурный	Диаграмма компонентов C4 и реестр ADR	Показывает модули хаба (приём сканов, трекинг, маршрутизация) и фиксирует принятые решения	PlantUML и Markdown в папке /docs репозитория
Контрактный	Спецификация openapi.yaml	Описывает API трекинга: POST /scans (скан QR/RFID), GET /parcels/{id}/track, POST /routes	OpenAPI 3.0.3, корень репозитория
Кодовый / тестовый	Исходный код и набор автотестов (unit и интеграционные)	Реализация модулей, проверка разбора QR/RFID и расчёта маршрута	Репозиторий Git, папки /src и /tests, запуск в CI
Управленческий	Backlog задач и протокол код-ревью	Список работ по спринтам, статус задач, итоги проверки изменений	Доска задач и шаблон pull request

Связь артефактов: контракт OpenAPI определяет, что должны делать сервисы; код и тесты проверяют соответствие контракту; архитектурные документы объясняют, почему система устроена именно так; управленческие артефакты показывают, кто и что делает.

2 ПРОТОКОЛ РАБОТЫ С GIT И CODE REVIEW

2.1 Протокол Git

Команда использует упрощённую модель GitHub Flow [1]: ветка main всегда рабочая, новая работа ведётся в отдельных ветках. Непосредственные коммиты в main запрещены (защищённая ветка).

Таблица 3 – Именование веток

Ветка	**Правило имени**	**Пример**
main	Основная, стабильная, защищена от прямой записи	main
feature/*	feature/<модуль>-<суть>, новая функция или модуль	feature/rfid-scan-parser
bugfix/*	bugfix/<номер задачи>-<суть>, исправление ошибки	bugfix/142-qr-empty-code

Сообщения коммитов пишутся по схеме «тип: суть» (feat, fix, docs, test) [2], например «feat: разбор RFID-метки».

Условие слияния нового модуля: pull request из feature-ветки в main принимается, только если (1) все автотесты и сборка в CI прошли успешно; (2) есть минимум одно одобрение (approve) от ревьюера, который не является автором; (3) ветка актуальна относительно main и не имеет конфликтов; (4) к изменению приложены тесты и обновлён openapi.yaml, если менялся API [3].

2.2 Чек-лист Code Review

Ревьюер проверяет изменения по трём обязательным пунктам [4].

Таблица 4 – Обязательный чек-лист код-ревью

№	**Пункт проверки**	**Что именно смотрим**
1	Безопасность и корректность входных данных	Данные QR/RFID проверяются на пустоту, длину и формат; в коде нет паролей и токенов; запросы к БД параметризованы
2	Тесты	Для нового модуля есть автотесты на нормальный и ошибочный случай (повреждённая метка, дубль скана); все тесты проходят
3	Соответствие контракту и читаемость	Ответы совпадают с openapi.yaml; имена понятные; нет дублирования кода и «лишнего» кода

3 АРХИТЕКТУРНОЕ РЕШЕНИЕ ADR-001

Запись ADR фиксирует важное архитектурное решение вместе с причинами [5], чтобы через время команда помнила, почему выбрано именно так [6].

Таблица 5 – ADR-001

Поле	**Содержание**
Название	ADR-001. Передача событий сканирования через очередь сообщений
Статус	Принято
Проблема	Сканеры на складах хаба работают в зоне нестабильной сети и при пиковой нагрузке (разгрузка фуры) отправляют сотни сканов в минуту. При прямых синхронных вызовах сервиса трекинга события теряются при обрыве связи, а перегрузка сервиса замедляет работу сканеров.
Решение	Сканер отправляет событие в очередь сообщений (RabbitMQ) с подтверждением доставки. Сервис трекинга читает очередь в своём темпе и сохраняет события в БД. При отсутствии сети сканер накапливает события локально и отправляет их позже [7].
Отклонённая альтернатива	Синхронный REST-вызов POST /scans от каждого сканера напрямую в сервис трекинга.
Почему отклонена	Нет гарантии доставки при обрыве сети (событие теряется), сервис трекинга приходится рассчитывать на пиковую нагрузку, а любой его сбой сразу останавливает приёмку на складе. Очередь сглаживает пики и обеспечивает гарантированную доставку [8].
Последствия	Плюсы: нет потерь событий, независимость сканеров от сервиса. Минусы: нужно поддерживать брокер сообщений, статус скана в трекинге обновляется с небольшой задержкой.

ЗАКЛЮЧЕНИЕ

Составлен паспорт артефактов и протоколов проекта для АИС «Логистический хаб»: определены четыре артефакта (архитектурный, контрактный, кодовый и управленческий), описан протокол работы с Git с правилами именования веток и условиями слияния, составлен чек-лист код-ревью из трёх пунктов и оформлено решение ADR-001.

Единые правила и документы позволяют команде работать согласованно, быстро находить причины решений и снижают число ошибок при добавлении новых модулей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. GitHub Docs. GitHub flow [Электронный ресурс]. – URL: https://docs.github.com/en/get-started/using-github/github-flow (дата обращения: 30.10.2026).

2. Conventional Commits 1.0.0 [Электронный ресурс]. – URL: https://www.conventionalcommits.org/ru/v1.0.0/ (дата обращения: 30.10.2026).

3. GitHub Docs. About protected branches [Электронный ресурс]. – URL: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches (дата обращения: 30.10.2026).

4. Google Engineering Practices. Code Review Developer Guide [Электронный ресурс]. – URL: https://google.github.io/eng-practices/review/ (дата обращения: 30.10.2026).

5. Nygard M. Documenting Architecture Decisions [Электронный ресурс]. – URL: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions (дата обращения: 30.10.2026).

6. Architectural Decision Records. ADR GitHub organization [Электронный ресурс]. – URL: https://adr.github.io (дата обращения: 30.10.2026).

7. RabbitMQ. Reliability Guide [Электронный ресурс]. – URL: https://www.rabbitmq.com/docs/reliability (дата обращения: 30.10.2026).

8. Hohpe G., Woolf B. Enterprise Integration Patterns: Guaranteed Delivery [Электронный ресурс]. – URL: https://www.enterpriseintegrationpatterns.com/patterns/messaging/GuaranteedMessaging.html (дата обращения: 30.10.2026).